

Relatório – Processador RISC-V RV32I Single Cycle

Trabalho: Desenvolvimento de um Processador RISC-V RV32I Single Cycle

Aluno: Jones Márcio Nambundo

1. Obejtivo

O objetivo deste trabalho foi desenvolver um processador RISC-V RV32I single-cycle utilizando Verilog, implementando os principais módulos do caminho de dados e controle necessários para executar instruções básicas do conjunto RV32I.

2. RISC-V RV32I Single-Cycle

O RISC-V é uma arquitetura de conjunto de instruções (ISA – *Instruction Set Architecture*) aberta e modular, desenvolvida com o objetivo de fornecer uma plataforma simples, flexível e eficiente para implementação de processadores. Diferentemente de arquiteturas proprietárias, o RISC-V possui especificação aberta, permitindo que universidades, pesquisadores e empresas desenvolvam seus próprios processadores sem custos de licenciamento.

A arquitetura **RISC-V** segue o conceito **RISC (*Reduced Instruction Set Computer*)**, baseado em um conjunto reduzido de instruções simples e eficientes. Esse modelo busca simplificar o hardware, melhorar o desempenho e facilitar o desenvolvimento de processadores. Dentro da arquitetura RISC-V, o conjunto RV32I representa a versão base de 32 bits. O termo:

- **RV** significa RISC-V;
- **32** indica largura de dados de 32 bits;
- **I** representa o conjunto inteiro básico de instruções (*Integer Base Instruction Set*).

O conjunto RV32I inclui instruções fundamentais para:

- operações aritméticas;
- operações lógicas;
- acesso à memória;
- desvios condicionais;
- controle de fluxo do programa.

A implementação desenvolvida neste trabalho utiliza a arquitetura **Single-Cycle** (ciclo único). Nesse modelo, cada instrução é executada completamente em apenas um ciclo de clock. Assim, todas as etapas da execução — busca da instrução, decodificação, leitura dos registradores, execução na ALU, acesso à memória e escrita do resultado — ocorrem dentro do mesmo ciclo.

A principal vantagem da arquitetura single-cycle é sua simplicidade de implementação e entendimento, sendo amplamente utilizada no ensino de arquitetura de computadores e sistemas digitais. Além disso, facilita a visualização do datapath e dos sinais de controle do processador. Entretanto, como todas as instruções precisam ser concluídas em um único ciclo, o período de clock deve ser suficientemente grande para acomodar a instrução mais lenta do processador. Isso reduz o desempenho quando comparado a arquiteturas pipeline. O processador desenvolvido neste projeto implementa um datapath monociclo baseado na arquitetura RV32I, contendo os principais componentes de um processador RISC-V. O fluxo de execução do processador segue as etapas:

1. Busca da instrução na memória;
2. Decodificação da instrução;
3. Leitura dos registradores;
4. Geração de imediato;
5. Execução da operação na ALU;
6. Acesso à memória de dados;
7. Escrita do resultado no banco de registradores;
8. Atualização do Program Counter.

A implementação foi realizada em Verilog HDL e validada por meio de simulação utilizando testbench e visualização de formas de onda no GTKWave.

3. Módulos Implementados

- **Program Counter (PC):** Responsável por armazenar o endereço da instrução atual com suporte a PC+4, branch, jump e reset.
- **Instruction Memory:** Memória responsável pelo armazenamento das instruções carregadas por **arquivo .mem**.
- **Immediate Generator:** Geração dos imediatos dos tipos I, S, B, U e J.
- **Register File:** Banco de 32 registradores de 32 bits mantendo x0 sempre igual a zero.
- **ALU:** Implementação das operações add, sub, and, or, xor e slt.
- **ALU Control:** Controle das operações da ALU utilizando funct3, funct7 e ALUOp.
- **Main Controller:** Responsável pela geração dos sinais de controle do processador.
- **Data Memory:** Memória utilizada pelas instruções lw e sw.
- **Branch Unit:** Implementação das instruções beq e bne.
- **Top Level:** Integração completa do processador.
- **Testbench:** Responsável pela simulação, geração de clock, reset e visualização dos resultados.

4. Programa de testes e os resultados.

Para validação do processador RV32I single-cycle foi desenvolvido um programa de teste individualizado, conforme especificado no enunciado da atividade. O programa teve como objetivo verificar o funcionamento das operações aritméticas, lógicas, acesso à memória, desvio condicional (*branch*) e instruções de salto (*jump*). De acordo com a proposta da atividade, foram utilizadas as seguintes expressões para cálculo das constantes:

$$\begin{aligned}A &= N + 4 \\ B &= N + 2 \\ C &= 4 \times N \\ \text{MEM_ADDR} &= 4 \times N.\end{aligned}$$

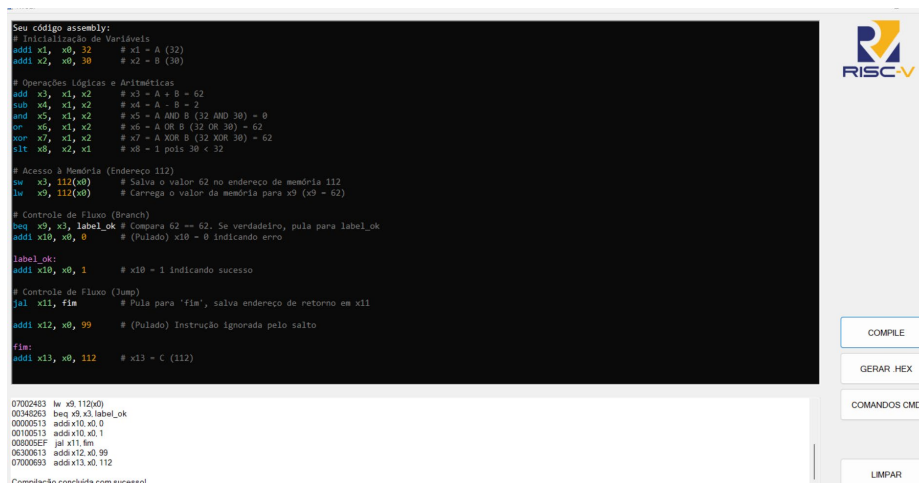
Neste trabalho foi utilizado o valor:

$$N = 28$$

A partir desse valor foram obtidas as seguintes constantes:

$$\begin{aligned}A &= 32 \\ B &= 30 \\ C &= 112 \\ \text{MEM_ADDR} &= 112\end{aligned}$$

O endereço MEM_ADDR foi utilizado nas instruções sw e lw para validar o funcionamento da memória de dados do processador. Já os valores de A e B foram utilizados nas operações aritméticas e lógicas executadas pela ALU. O valor C foi utilizado na etapa final do programa para validar a escrita correta nos registradores.



```
Seu código assembly
# Inicialização de Variáveis
addi x1, x0, 32 # x1 = A (32)
addi x2, x0, 30 # x2 = B (30)

# Operações Lógicas e Aritméticas
add x3, x1, x2 # x3 = A + B = 62
sub x4, x1, x2 # x4 = A - B = 2
and x5, x1, x2 # x5 = A AND B (32 AND 30) = 0
or x6, x1, x2 # x6 = A OR B (32 OR 30) = 62
xor x7, x1, x2 # x7 = A XOR B (32 XOR 30) = 62
slt x8, x2, x1 # x8 = 1 pois 30 < 32

# Acesso à Memória (Endereço 112)
sw x3, 112(x0) # Salva o valor 62 no endereço de memória 112
lw x9, 112(x0) # Carrega o valor da memória para x9 (x9 = 62)

# Controle de Fluxo (Branch)
beq x9, x3, label_ok # Compara 62 == 62. Se verdadeiro, pula para label_ok
addi x10, x0, 0 # (Pulado) x10 = 0 indicando erro

label_ok:
addi x10, x0, 1 # x10 = 1 indicando sucesso

# Controle de Fluxo (Jump)
jal x11, fim # Pula para 'fim', salva endereço de retorno em x11
addi x12, x0, 99 # (Pulado) Instrução ignorada pelo salto

fim:
addi x13, x0, 112 # x13 = C (112)

07002483 lw x9, 112(x0)
0340263 beq x9, x3, label_ok
0000513 addi x10, x0, 0
00100513 addi x10, x0, 1
000005EF jal x11, fim
06300613 addi x12, x0, 99
07000693 addi x13, x0, 112
Compilação concluída com sucesso!
```

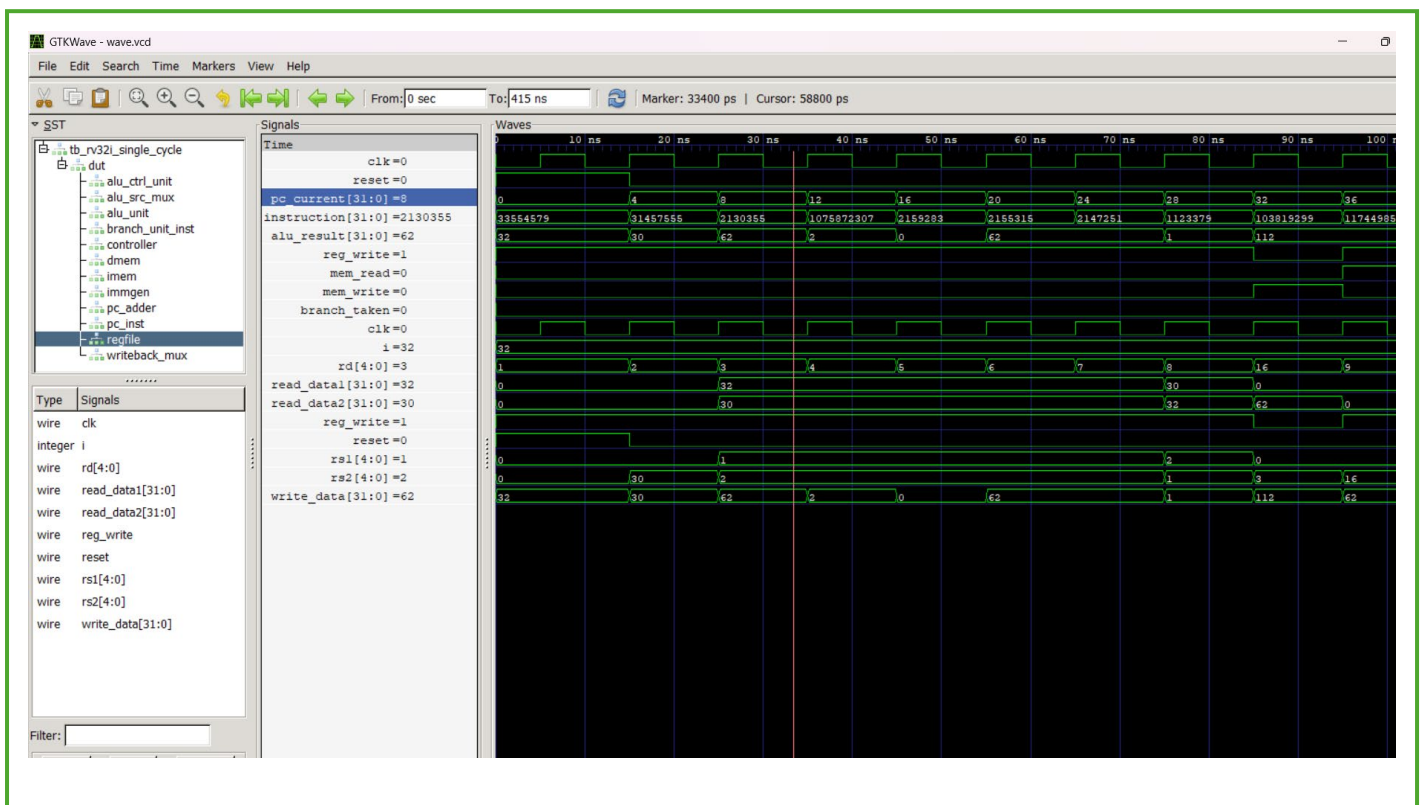
Durante a simulação do processador RV32I single-cycle foi possível verificar o correto funcionamento das instruções de memória, branch e jump. A instrução lw x9, 112(x0) realizou corretamente a leitura do valor armazenado na memória de dados, carregando o valor 62 para o registrador x9. Em seguida, a instrução beq x9, x3, label_ok comparou os registradores x9 e x3. Como ambos possuíam o mesmo valor,

operandos utilizados pela ALU (*alu*). Dependendo do tipo da instrução, os dados podem passar pelos multiplexadores (*mux2* e *mux4*), permitindo selecionar entre registradores, imediatos, memória ou PC+4.

A ALU executa as operações aritméticas e lógicas do processador, enquanto o módulo *branch_unit* verifica as condições de desvio condicional (*beq* e *bne*). Para instruções de acesso à memória, o módulo **data_memory** realiza operações de leitura (*lw*) e escrita (*sw*). O resultado é enviado novamente ao banco de registradores através do caminho de *write-back*, enquanto a lógica de controle atualiza o próximo valor do *Program Counter*, permitindo a execução contínua do programa. O esquemático RTL confirma a correta integração dos módulos implementados e representa o funcionamento completo do processador RV32I single-cycle desenvolvido neste trabalho.

6. Análise Detalhada das Formas de Onda no GTKWave

As Figuras apresentadas mostram as formas de onda geradas durante a simulação do processador RV32I single-cycle no GTKWave. Através delas foi possível acompanhar detalhadamente a execução das instruções, a evolução do Program Counter, os resultados produzidos pela ALU e os valores escritos no banco de registradores.



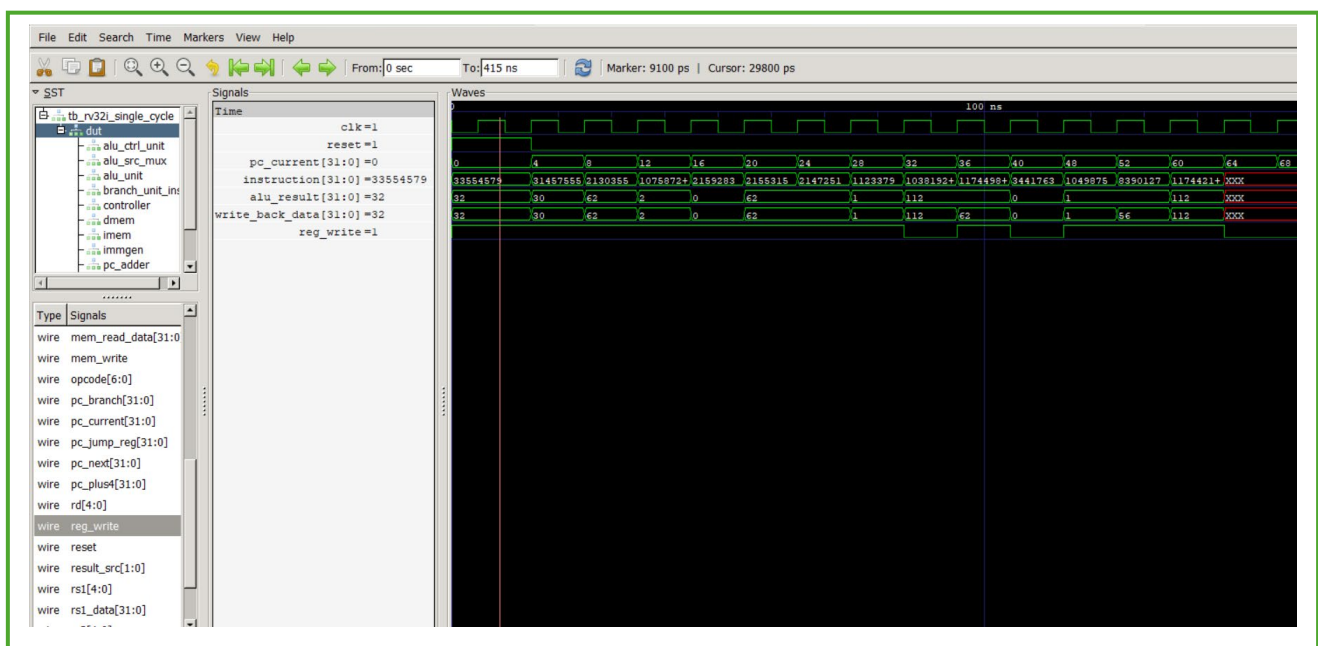
O sinal *clk* representa o *clock* do sistema e controla a sincronização de todas as operações do processador. A cada borda de subida do clock uma nova instrução é executada. Já o sinal *reset* é utilizado para inicializar o sistema, fazendo com que o Program Counter inicie no endereço 0, garantindo o início correto da execução do programa.

Um dos principais sinais observados é o `pc_current`, que representa o valor atual do Program Counter (PC). Durante a execução do programa é possível observar que o PC evolui sequencialmente nos valores:

0 → 4 → 8 → 12 → 16 → 20 → 24 → 28 → 32 ...

Esse incremento de 4 em 4 ocorre porque cada instrução da arquitetura RV32I possui tamanho fixo de 32 bits (4 bytes). Assim, após a execução de cada instrução, o processador atualiza automaticamente o PC para o próximo endereço de memória.

O sinal ***instruction*** mostra as instruções carregadas da memória de instruções em cada ciclo. Através dele é possível acompanhar a sequência completa do programa assembly implementado para validação do processador.



Os sinais `alu_result` e `write_back_data` demonstram os resultados produzidos pela ALU e os valores escritos de volta no banco de registradores. Durante a simulação foram observados os seguintes resultados principais:

- Valor 32, correspondente à instrução: **`addi x1, x0, 32`**

Nessa operação, o registrador x1 recebe o valor 32, correspondente à constante A.

- Valor 30, correspondente à instrução: **`addi x2, x0, 30`**

Nessa etapa, o registrador x2 recebe o valor 30, correspondente à constante B.

- Valor 62, produzido pela instrução: **`add x3, x1, x2`**

A ALU realiza a soma entre os valores de x1 e x2: **`32 + 30 = 62`**

Esse resultado é armazenado no registrador x3, sendo posteriormente utilizado nas instruções de memória e branch. Também é possível observar outros resultados importantes da ALU, como:

- 2 na operação de subtração ($32 - 30$);
- 0 na operação lógica AND;
- 1 na instrução `slt`, indicando que $30 < 32$.

Os sinais *mem_read* e *mem_write* mostram o funcionamento correto das instruções de acesso à memória (`lw` e `sw`). Durante a execução da instrução `sw`, o valor 62 é armazenado no endereço de memória 112. Posteriormente, na instrução `lw`, esse mesmo valor é carregado para o registrador x9, validando corretamente a memória de dados do processador. Outro sinal importante observado é `branch_taken`, utilizado pela unidade de branch. Durante a execução da instrução: **`beq x9, x3, label_ok`**

O processador compara os valores dos registradores x9 e x3. Como ambos possuem o valor 62, o branch é ativado corretamente, alterando o fluxo do programa para o rótulo `label_ok`. Além disso, a simulação também valida o funcionamento da instrução `jal`, onde o Program Counter é atualizado para um novo endereço, realizando corretamente o salto para a instrução final do programa.

As formas de onda apresentadas confirmam o correto funcionamento do processador RV32I single-cycle implementado, demonstrando que os módulos de controle, ALU, memória e banco de registradores operaram de forma integrada e coerente durante toda a execução do programa de teste.